Image Preprocessing for Person Re-Identification

1. Objective

The aim of this assignment is to apply image preprocessing techniques to enhance images in a person re-identification dataset. Preprocessing improves image quality and helps downstream tasks, such as feature extraction and recognition.

2. Dataset Description

- Dataset contains images of different persons captured from multiple angles.
- Organized in 4 main categories:
    1. person_with_bag
    2. person_without_bag
    3. both_large
    4. both_small
- Each category has subfolders: bounding_box_train, bounding_box_test, and query.
- Images vary in size, illumination, and viewpoint, requiring preprocessing to normalize and enhance features.

Preprocessing Techniques Used

3.1 Histogram Equalization

Purpose: Enhances global contrast by spreading intensity values evenly.

Effect: Makes dark or poorly illuminated areas more visible.

Code Snippet:

```
enhanced = cv2.equalizeHist(gray)
```

3.2 Gaussian Filtering (Spatial Smoothing)

Purpose: Reduces image noise and smooths textures.

Effect: Preserves main structures while removing high-frequency noise.

Code Snippet:

```
smoothed = cv2.GaussianBlur(enhanced, (3,3), 0)
```

3.3 Sharpening Filter

Purpose: Enhances edges and fine details.

Effect: Highlights unique features like clothing, bags, and patterns.

Code Snippet:

```
kernel = np.array([[0,-1,0], [-1,5,-1], [0,-1,0]])
sharpened = cv2.filter2D(smoothed, -1, kernel)
```

4. Preprocessing Workflow

1. Resize each image to a fixed size (128 x 256) for consistency.
2. Convert to grayscale for uniform intensity processing.
3. Histogram equalization to improve global contrast.
4. Gaussian filter to reduce noise.
5. Sharpening filter to enhance edges.
6. Save processed images in a separate folder, preserving original folder structure.

5.Implementation (Python + OpenCV)

Preprocessing Code

```
import cv2
import os
import numpy as np

# Paths
input_parent = r"F:\WB_WoB-ReID"        # original dataset
output_parent = r"F:\processed_dataset"   # processed images

os.makedirs(output_parent, exist_ok=True)

# Number of images per category to process
IMAGES_PER_CATEGORY = 3

# Preprocessing function
def preprocess_image(image):
    # Resize
    image = cv2.resize(image, (128, 256))

    # Convert to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # -------- Technique 1: Histogram Equalization --------
    enhanced = cv2.equalizeHist(gray)

    # -------- Technique 2: Gaussian Filtering (Spatial) --------
    smoothed = cv2.GaussianBlur(enhanced, (3,3), 0)

    # -------- Technique 3: Sharpening (Spatial) --------
    kernel = np.array([[0,-1,0], [-1,5,-1], [0,-1,0]])
    sharpened = cv2.filter2D(smoothed, -1, kernel)
```

```
    return sharpened

# ---------- Processing ----------
for category in os.listdir(input_parent):
    category_path = os.path.join(input_parent, category)

    if not os.path.isdir(category_path):
        continue

    print(f"Processing category: {category}")

    out_category_path = os.path.join(output_parent, category)
    os.makedirs(out_category_path, exist_ok=True)

    count = 0  # reset per category

    for subfolder in os.listdir(category_path):
        subfolder_path = os.path.join(category_path, subfolder)

        if not os.path.isdir(subfolder_path):
            continue

        out_subfolder_path = os.path.join(out_category_path, subfolder)
        os.makedirs(out_subfolder_path, exist_ok=True)

        for filename in os.listdir(subfolder_path):
            if filename.lower().endswith(('.jpg', '.png', '.jpeg')):

                if count >= IMAGES_PER_CATEGORY:
                    break

                img_path = os.path.join(subfolder_path, filename)
                image = cv2.imread(img_path)

                if image is None:
                    continue

                processed = preprocess_image(image)

                save_path = os.path.join(out_subfolder_path, filename)
                cv2.imwrite(save_path, processed)

                count += 1

        if count >= IMAGES_PER_CATEGORY:
            break

print("✅ Enhanced spatial preprocessing completed!")
```

OUTPUTS:


without_bag - Original (Left) vs Processed (Right)


with_bag - Original (Left) vs Processed (Right)